Exploring Hierarchical Scripts with Story Generation

Jamie C Macbeth[1] JMACBETH@SMITH.EDU
Boming Zhang[2] BOMINGZHANG@UMASS.EDU
Nhan Trong Pham[2] TRONGNHANPHA@UMASS.EDU
Zishu (Anna) Zhao[1] AZHAO93@SMITH.EDU
Ruixi Liang[1] ALIANG@SMITH.EDU
Mark Roberts[3] MAKRO@ICONIUMLABS.COM

[1]Smith College, Northampton, MA, USA
[2]University of Massachusetts, Amherst, MA, USA
[3]Iconium Labs, Tempe, AZ, USA

Abstract

Traditionally scripts were used for natural language understanding and story understanding systems, allowing them to perform inference and reasoning in highly stereotyped domains of social activity. The idea of hierarchy has been explored sporadically in prior work on scripts, and recent work suggests the possibility of scripts being more hierarchical than previously imagined. In this paper, we claim that scripts can be truly hierarchical. We present a modified version of the Script Applier Mechanism called the Hierarchical Script Maincon Story Generator, a story generator that will serve as a model of our hierarchical script theory. We also construct a hierarchical version of the restaurant script to use for story generation. We generate stories from the hierarchical scripts, using story generation as a tool for investigating the hierarchical nature of scripts and for evaluating hierarchical script theory.

1. Introduction

Traditionally scripts were used for natural language understanding and story understanding systems (Cullingford, 1977b, 1979) allowing them to perform inference and reasoning in highly stereotyped and predictable domains of social activity. Although some work has attempted to broaden the applicability of scripts, the connections between scripts and other knowledge representation and reasoning systems remain poorly understood. Broadening the kinds of cognitive tasks which scripts might be used for is an area of active research.

Recent work has explored whether scripts and their applicability might be expanded beyond story understanding towards story generation (McKenzie et al., 2022; Cardona-Rivera et al., 2024). In story understanding, scripts contain mundane knowledge which is needed in allowing the understander to fill in unstated details which were elided or not mentioned. However, story generation from scripts may be more difficult than it would appear to be. Here is an example of a story which

could be generated by constructing a story representation which randomly samples events from the ordered sequences in the well-known restaurant script given in Schank & Abelson (1977):

> "Emily walked into Amelie. She looked over at the tables. She picked up a menu and the cook made Cog Au Vin. Then the server walked over to Emily and Emily gave money to the cashier."

Although it may be possible to make out what happened, this unnatural story telling example makes plain the centrality and importance of some events in the script structure in relation to others when it comes to both story telling and understanding.

The idea of hierarchy has been explored sporadically in prior work on scripts (Cullingford, 1977b; Lehnert, 1975). Recent work and commentaries suggest both the possibility of scripts being more hierarchical than previously imagined (Macbeth et al., 2025a) and their potential use for story generation (Cardona-Rivera et al., 2024). In this paper, we claim that scripts can be truly hierarchical structures. As evidence, we present *hierarchical maincon scripts* as well as a modified version of the Script Applier Mechanism called the *Hierarchical Maincon Script Story Generator*. We also construct a hierarchical version of the restaurant script to use for story generation. We then generate stories from the hierarchical script and use them as a vehicle for exploring the hierarchical nature of scripts and for evaluating hierarchical script theory.

The remainder of this paper is laid out as follows: Section 2 covers background on scripts, the Script Applier Mechanism, and previous work on the hierarchical nature of scripts. The hierarchical restaurant script is presented in Section 3. Sections 4 and 5 discuss the Hierarchical Maincon Script Story Generator and provide example generated stories and discussion of them. Sections 6 and 7 discuss related work and conclude the paper.

2. Background

2.1 Scripts

Scripts (Schank & Abelson, 1977) are a representational construct which enables cognitive systems to encode culturally shared knowledge of socially ritualized activities. Scripts consist of abstract and generalized event patterns arising from encountering similar social situations repeatedly. In the original literature (Schank & Abelson, 1977), scripts were represented as chains of event patterns called *episodes* which included well-defined roles called *variables*. Episodes prescribed the order in which things happen, and individual events frequently result in states of the world which enable subsequent events and states in the chain. Thus episodes were also referred to as "event chains" or "causal chains" (Schank & Abelson, 1977).

Scripts are also a general theory of inference and reasoning processes in the form of pattern matching, conceptual structure instantiation, and role-filling that serve as an alternative to logical formulations for representing human thinking. Scripts are *applied* to perform reasoning in a language-free match-and-instantiate process. Script-based systems perform inferences by matching one or more language-free input event conceptualizations against conceptualizations in an script's event chain episode, binding and filling script variables, and realizing conceptualizations from the episode which were not part of the initial input. Scripts are also viewed as a form of knowledge

and memory organization. Intuitively, deep understanding should draw from an large volume of knowledge from very different domains, but, at many moments in the process, accessing the particular pieces of knowledge that are needed should not require an inordinate amount of search or open inference.

2.2 SAM

The Script Applier Mechanism (SAM, Cullingford, 1975, 1977a,b, 1979) was a cognitive system for natural language story understanding which implemented scripts to model human story understanding on a variety of topics, from official diplomatic visits to oil spills. SAM demonstrated its depth of understanding by providing summaries of stories at varying levels of detail and in different languages, and by answering questions about them. In this work, as a cognitive model of script theory, scripts were represented in language-independent conceptual dependency (CD) conceptual structures (Schank, 1972, 1973).

The basic mechanism of SAM is the following. First, the English Language Interpreter system processes a sentence of the story text, producing a language-independent conceptual dependency structure corresponding to the text through a process called conceptual analysis (Cullingford, 1977b). Then, at the center of SAM was a component called the Script Applier. Its main role was to receive the conceptual structure corresponding to a story event and to search for and record matches between the input conceptual structure and pattern structures in the script while binding script variables. A match also triggered the "invocation" of a particular script if a script was not already active. Next, event conceptualization patterns in the episodic chain of the active script which were not part of the input were instantiated and realized with the variables of the script filled in with their bindings. These instantiations form the basis for SAM's default reasoning capability. SAM repeated these steps, processing the story sentence by sentence and instantiating event chains through the episodes.

The original work on SAM and CD uses the term *picture producer* (or *PP*) to refer to story characters, objects, places, "props" and other "conceptual nominals" that persisted across multiple story conceptualizations and were assigned to one or more script variables. To address reference resolution issues associated with PPs, SAM created and managed system objects corresponding to PPs called *tokens*[1]. In addition to managing tokens for explicitly mentioned PPs, SAM instantiated "implicit reference" tokens and bound them to script variables in cases when the story input did not provide a PP, an additional aspect of the default reasoning capability provided by the script.

SAM could use the detailed story representations it built to answer questions posed about the narratives it read (Lehnert, 1975). Here is an example interaction with SAM after it read a restaurant story:

> Q. What happened when the hostess gave John a menu?
> A. John read the menu. The waiter saw that John was at the table. The waiter went to the table.

1. The term "token" in the script applier literature is not to be confused with the concept of tokens appearing in large language models and other currently mainstream artificial neural network-based natural language processing systems.

SAM was also capable of generating "long paraphrases" of stories it read. These included surface realizations of all of the conceptualizations in the story representation created in the script application process, including those which it filled in or inferred. Below is an excerpt from a text generated by SAM from Cullingford (1975):

> ... [John] entered Leone's. He looked around inside it. He saw he could go to a table. He went to it. He sat down in the seat. He ordered some lasagna. The waiter indicated to the chef John would like him to prepare something. The chef prepared the lasagna. The waiter got it from the chef. The waiter went to the table. He served the lasagna to John. He ate it. He became full ...

In the current work, SAM is inverted into a story generation system. SAM's script applier and tokenization features are used to create story representations consisting of non-linguistic CD conceptual structures, and SAM's paraphrase generation capability realizes story representations in English.

2.3 Hierarchies within Scripts

Although it is not frequently noted, early work on scripts did have high-level hierarchical organization of event chain episode structures. Schank & Abelson (1977) details the "coffee shop track" of the restaurant script, dividing it into four *scenes*. In the Entering scene, the customer enters to the restaurant, looks around, finds a place to sit, and sits. In the Ordering scene, the customer chooses an item from the menu and orders it with their server, who passes the order on to the cook. In the Eating scene, the server delivers the item to the customer, who eats it. The script concludes with the Exiting scene, in which the server provides a check to the customer, and the customer leaves a tip, pays the cashier, and leaves. Each scene could have multiple sequences of *causal chains* of acts and events called *subscenes*, representing multiple ways of completing the activity.

Scripts also had hierarchical references to other scripts in the original work (Cullingford, 1977b, 1979). For example, to understand stories where individuals took trips to visit museums or theaters or to eat at restaurants, SAM used "global scripts" and "script situations" in which a script at one level acted as an "instrument" for a script at the next level up (Cullingford, 1977b, page 29). For example, Cullingford's $TRIP global script contained references to the $RESTAURANT script and other scripts (such as $THEATER, $MUSEUM, and $VARIETYSTORE) as "segments" (Cullingford, 1977b, page 139).

In addition to long paraphrases and question answers, SAM could also generate summaries of stories which were were extremely short and to the point (Cullingford, 1975):

> John went to a restaurant and he ate a lobster.

SAM used a special conceptual structure called a *maincon* which was built into the script to generate summaries. The maincon was a CD structure to represent the main or most important act of the script or of a script scene and it contained script variables which were bound in the script application process which could be used to generate a summary. SAM also used the scenes' maincons alongside the goal-subgoal structure that runs through a script's scene hierarchy to answer goal-oriented questions (Lehnert, 1978). For example, asking "why did John order lasagna" invoked the maincon

of the Ordering scene, while the answer "because he wanted to eat lasagna" draws on the maincon of the eating scene, reflecting how the goal behind one scene's act (ordering) is the maincon of the next scene (eating).

The case for hierarchical representations of scripts is also well-supported by experimental psychology. Scripts are "represented in memory as hierarchically and temporally organized information packets ... arranged in such a way that useful generalizations and predictions are available to aid people in understanding events" (Abbott et al., 1985). "The script is not a linear chain of actions at one level but rather a hierarchically organized 'tree' of events with several levels of subordinate actions" (Bower et al., 1979).

In recent work, Macbeth et al. (2025a) propose that scripts are hierarchical and are plan- and goal-oriented as a flexible and reusable cognitive representation, encoding the restaurant script in a planning language that natively supports modeling in Python. Their analysis suggests that almost every action in the restaurant script is associated with an underlying goal, motivating the authors to represent the script as a hierarchical goal network. Their results suggest that, viewed as a decomposition tree from a hierarchical goal network planning process, the restaurant script may have more levels of hierarchy than just the "scene" level.

3. A Hierarchical Restaurant Script

In this section we present an attempt to build script structures which maximize the degree of hierarchical structure, which we call *hierarchical maincon scripts*. Like scripts in their original conception, hierarchical maincon scripts consist of language-free event conceptualizations containing bindable script variables. However, while traditional scripts are chains of events with occasional hierarchical links between them, hierarchical maincon scripts form a tree structure of event conceptualizations with internal nodes having an ordered list of children nodes. All nodes of the tree have a CD conceptualization; terminal nodes have a CD conceptualization which, in "flat" scripts, would have been part of an episodic event pattern chain. Internal nodes in the tree have a structural function in that they store a conceptualization which represents the maincon or most important event pattern of all terminal nodes in the subtree rooted at that node.

We hand-built a hierarchical maincon tree structure for the restaurant script by enhancing the traditional script structure, adding hierarchical levels using our commonsense knowledge of the importance of events and script acts. For simplicity, in the discussion and figures that follow, we allow English glosses to stand in for the language-free CD structures of script nodes. In these English glosses, C and F stand in for the customer and the "food item" script variables respectively[2]. W is an abbreviation for the waiter or server, which appeared as a role in the original script but is a fixed picture producer in our implementation. A table mapping the CD structures from Schank & Abelson (1977) (e.g. "C MTRANS food list to CP(C)") to English glosses is provided in the appendix of this paper.

Figure 1 shows the top two levels of our hierarchical maincon restaurant script. For each node, the node label is an English gloss of the maincon conceptualization, or most important act of the subtree rooted at each node. The maincon of the entire script is the act of the customer eating the

2. This is a slight departure from the notation in Schank & Abelson (1977), which uses S to represent the customer.

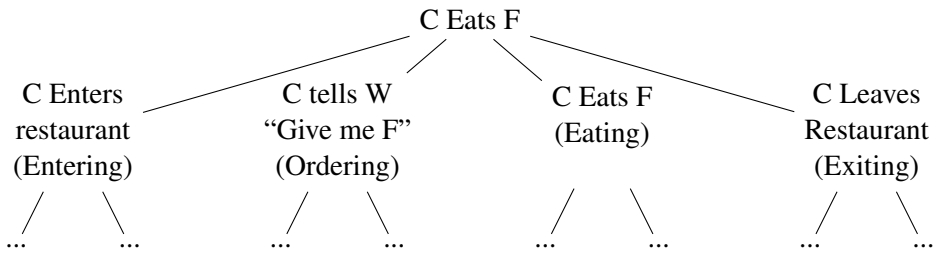


Figure 1: The top two levels of a hierarchical maincon script version of the restaurant script. For each node, the node label is an English gloss of the maincon conceptualization, or most important act of the subtree rooted at each node. The four children of the root node correspond to the Entering, Ordering, Eating, and Exiting scenes of the original restaurant script. C, W, and F stand in for the customer, the waiter, and the "food item" respectively.

food that they ordered. The four children of the root node are the roots of subtrees correspond to the Entering, Ordering, Eating, and Exiting scenes of the original restaurant script (Schank & Abelson, 1977), and their maincon conceptualizations are for the customer entering the restaurant, ordering a food item, eating that food item, and leaving the restaurant.

To determine the hierarchical tree structure from the restaurant script, we followed a similar procedure to that used in prior human participant-based studies of script knowledge (Abbott et al., 1985; Macbeth et al., 2025a). Assuming a strong link between the high-level goal of an act, and its importance in relation to events near it in a script's event chain, we informally asked ourselves a "why" question about the goal of each act in the script, and placed the script act that served as the answer to the question as the immediate parent of that act. For example, we asked ourselves "why does the customer read the menu," corresponding to the "C MTRANS food list to CP(C)" act described in the Ordering scene of the restaurant script as it appeared in Schank & Abelson (1977). Our answer was "So that they could choose a food item," corresponding to the "C MBUILD Choice of F" act. We then placed "C reads menu" as an immediate child of "C decides on F" in the tree. The process would continue by asking similar questions and placing other acts as subevents of the "C MTRANS food list to CP(C)" (for example, since the customer picks up the menu in order to read it, the "customer picks up the menu" node ended up a child node of the "customer reading menu" node).

We additionally required that any conceptualization at any internal node representing the maincon must have a corresponding leaf node in its subtree with the same conceptualization. This made children of each internal node look similar to a task decomposition method for accomplishing the goal of the parent. When an event node had multiple child subevent nodes, the list of children was ordered according to the causal chain order in which they appeared in the original restaurant script. In many cases, in order to maintain the original scene structures (Schank & Abelson, 1977), internal nodes had leaf node children whose CD conceptualizations were identical to the maincons of their immediate parents.

Figure 2 shows the subtree which we constructed corresponding to the Ordering scene of the restaurant script. The maincon of the root node, whose English gloss is "C tells W 'give me F,"' was

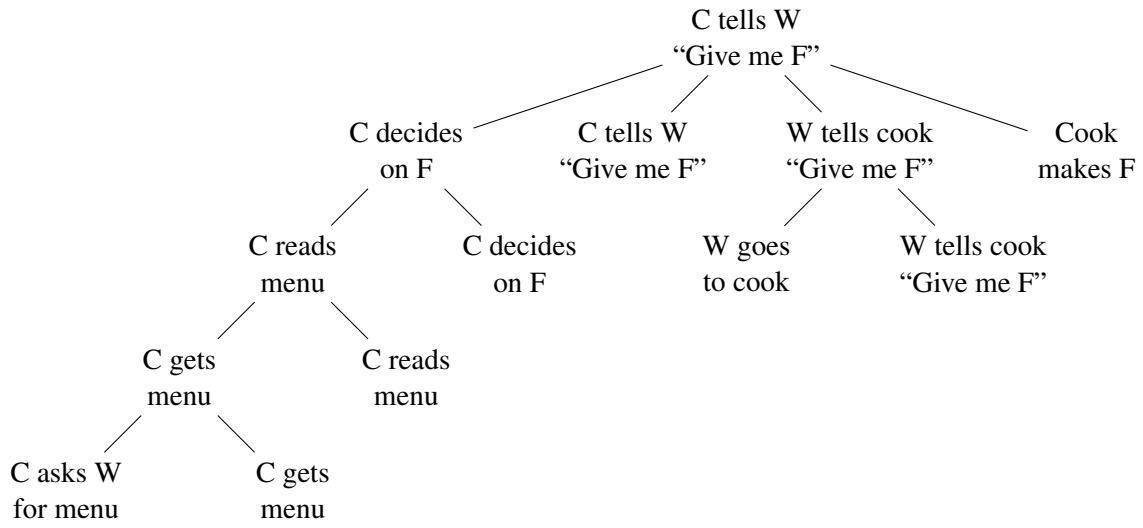


Figure 2: A subtree for the Ordering scene of the restaurant script. For each node, the node label is the maincon or most important act of the subtree rooted at each node. C, W, and F stand in for the customer, the waiter, and the “food item” respectively. Some paths present in the original Ordering scene (Schank & Abelson, 1977), such as paths for the situation in which the cook tells the waiter that the item that the customer ordered is not available and the full path for the waiter going and getting a menu for the customer are not visible.

found to be the most important act of the entire subtree, and the same conceptual pattern appears as a terminal node as an immediate child of the root node. The immediate children of the root node, “C decides on F,” “C tells W ‘give me F,”’, “W tells cook ‘give me F,”’ and “Cook makes F,” appear in temporal order and in the same order in which they appear in the causal chain in the original script (Schank & Abelson, 1977). The “C gets menu” pattern corresponds to a CD ATRANS act which is agnostic as to the giver and receiver, and could also appear in English as “W gives menu to C.” For simplicity, we left out certain paths present in the original Ordering scene (Schank & Abelson, 1977) related to the menu being available, namely the path where the waiter goes and gets a menu for the customer and the path where the menu is already on the table. We also did not include paths for the situation in which the cook tells the waiter that the item that the customer ordered is not available.

Figure 3 shows the subtree which we constructed corresponding to the Exiting scene. Although we chose “C leaves restaurant” to be the maincon of the root node, it was a difficult choice between it and “C pays cashier” to serve as the most important act of the entire subtree. We ended up choosing “C leaves restaurant” in part because the question “why does the customer pay the cashier,” has “because the customer wants to leave the restaurant” as a somewhat reasonable answer. At the same time, leaving a tip and paying the cashier intuitively are preconditions for leaving the restaurant for reasons lying outside of the single restaurant visit. For example, there may be serious consequences if the customer eats and leaves but does not pay their bill or does not leave a tip. As with the

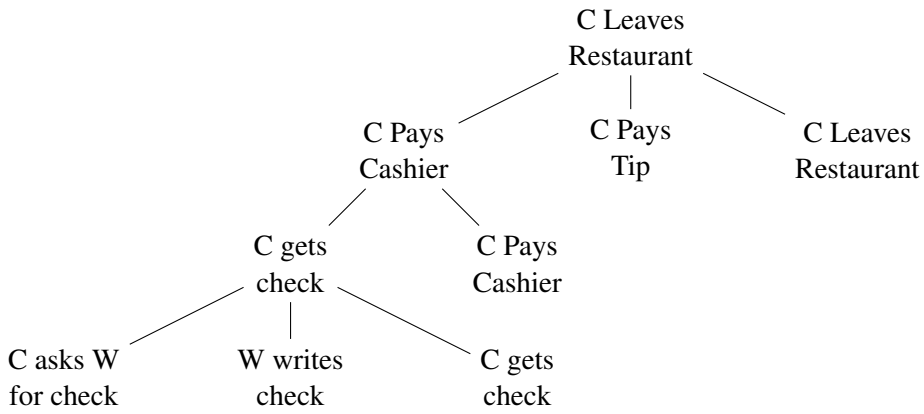


Figure 3: A subtree for the Exiting scene of the restaurant script. For each node, the node label is the maincon or most important act of the subtree rooted at each node. C, W, and F stand in for the customer, the waiter, and the "food item" respectively.

Ordering scene, the "C gets check" pattern corresponds to a CD ATRANS act which is agnostic as to the giver and receiver and could also appear in English as "W gives check to C."

Figure 4 shows the subtrees corresponding to the Entering and Eating scenes. For the Entering scene, which represents the customer entering the restaurant, looking at the tables, deciding where to sit, and sitting down, we chose "C Enters restaurant" as the maincon at the root node and the most important event of the subtree. Although "C Enters restaurant" appears to be more important than any other event in the scene, this is an awkward choice because, if we ask the question "why did the customer sit down", "because the customer entered the restaurant" is not the most intuitive answer.

In total, our hierarchical restaurant script structure has six levels of hierarchy, and consists of 22 leaf nodes and 14 interior nodes for a total of 36 nodes. By comparison, Schank & Abelson's (1977) original description of the restaurant script has 33 CD acts.

4. Hierarchical Maincon Story Generation

To assess our theory and model of hierarchical maincon scripts, we modified a version of the script applier mechanism into a story generator called the *Hierarchical Maincon Script Story Generator* (HMSSG). The script story generator does roughly the opposite of the script story applier. Instead of applying the script to CD structures from an input story to bind the variables, HMSSG arbitrarily creates tokens and binds script variables to them. Then it creates a story representation and generates a natural language surface realization using SAM's "long paraphrase" feature. However, unlike the original SAM, it creates a story representation via a traversal of the script's tree structure.

4.1 Binding Script Variables

As a first step, The SAM story generator creates tokens corresponding to script variables. We used a small set of classes for picture producers which reflected the original SAM's simple ontology:

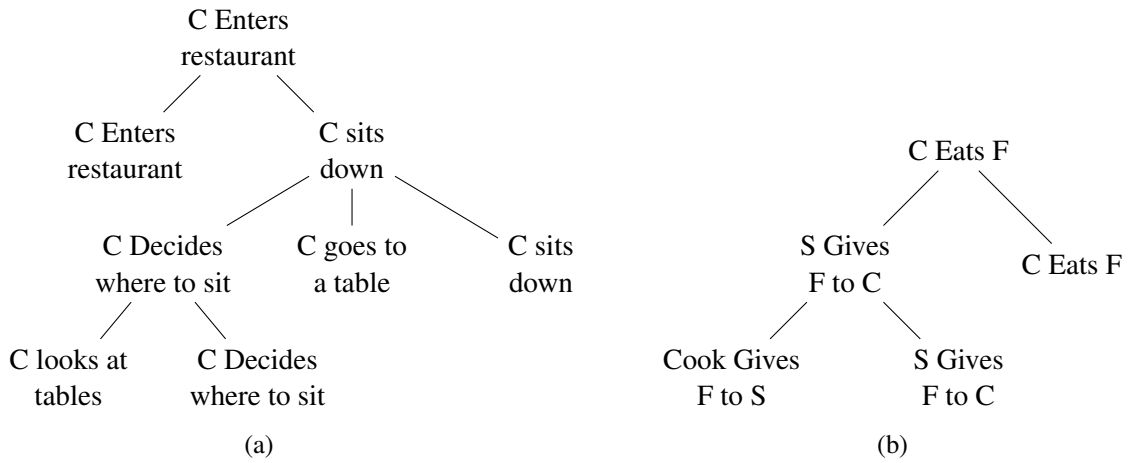


Figure 4: Subtrees for the Entering scene (a) and the Eating scene (b) of the restaurant script.

#ORGANIZATION, #PERSON, #LOCALE, #LINK (for roads and highways), #STRUCTURE (for structured objects), and #PHYSOBJ for other objects. For the story examples shown later in this paper based on the restaurant script, HMSSG randomly selects a customer name, the name of a restaurant, and a food item, generates tokens for them, and binds them to the customer, restaurant, and food item script variables.

Next, tokens are bound to script variables representing roles in the script. In SAM, script variables were structures containing knowledge about what kind of picture producers could be bound to the variable. For story generation, this knowledge is used to generate a token. The script variable structures in SAM also contain surface language expressions for how to refer to the role when generating summaries of stories that SAM read. In HMSSG this feature allows the story generator to generate a variety of surface language forms in reference to the same picture producer. For example, assuming that a token is created for a #PERSON named Ethan for the waiter variable, it can create CD structures for the story representation that will stochastically generate "Ethan", "the server", "Ethan, the server", or "him" downstream.

4.2 Building the Story Representation

Next the system instantiates a story representation from the script as an ordered list of CD structures. It serializes the hierarchical script structure, performing a tree traversal through it. Starting at the root of the tree and continuing at each interior node of the tree, the system chooses to either instantiate the "maincon" script pattern at that node into a story event or to proceed to expand and process the children of that node in order. If a leaf node is reached and processed, the script pattern of the leaf is instantiated into an event and added to the story representation.

HMSSG has two modes for deciding whether to instantiate a node or proceed to its children. It can either proceed deeper stochastically or it can proceed as deep as possible based on a maximum depth parameter. With hope, these features have the effect of generating a coherent story in which different parts of the story have varying degrees of detail.

(a)	(b)	(c)
Ryan went into Succotash and sat down. He decided on kebab and ordered it. The server told the cook to make kebab and the cook made it. The server gave kebab to Ryan. Ryan ate kebab, paid the cashier, paid a tip to the server, and left.	Samantha went to Dubrovnik. She asked the server for a menu. Samantha got a menu from the server and read it. She chose salmon and told the server to give them salmon. The server went to the cook and asked for salmon. The cook made salmon. The cook gave the server salmon and the server gave it to Samantha. She ate salmon and got out of Dubrovnik.	Ashley got into Hopleaf. They looked around at the tables, and decided on a table. They went to the table and sat down. Ashley ordered salad and ate it. They asked the waiter for the check. The waiter wrote the check and gave it to Ashley. Ashley paid the cashier, paid a tip to the server, and left Hopleaf.

Table 1: Example stories from HMSSG on the hierarchical restaurant script. Story (a) is a result from running HMSSG with a maximum depth of three. Stories (b) and (c) are generated from choosing randomly to go deeper at each interior node.

4.3 Generating the Story

Lastly, a conceptual generator system which can create surface language realizations from language-free CD structures is fed each of the story representation CD structures in turn to generate an English story. Table 1 provides examples of stories generated by the HMSSG system.

4.4 Implementation

The HMSSG system is implemented in Common Lisp using a mixture of original and new code. It uses code from a Common Lisp implementation of MicroSAM (also known as McSAM), a "very limited script applier" which appeared in Schank & Riesbeck (1981). HMSSG uses MicroSAM's functions for instantiating story events by replacing script variables in script patterns given a list of token bindings to script variables. Like the original SAM, we used Neil Goldman's BABEL system to generate English realizations (Goldman, 1975). Our version of BABEL from the Stanford AI Project's SAILDART archives[3] dates from 1973, predating the work on SAM at Yale which culminated in the late 1970s. We translated our 1973 version of BABEL to Common Lisp and enhanced it with discrimination nets for the CD ATTEND, MBUILD, and MOVE primitives and several word senses. We also implemented new code for hierarchical scripts and for generating a language-free story representation from them.

3. https://www.saildart.org

5. Discussion

We generally find that the stories generated by HMSSG (Table 1) are more natural-sounding than stories that randomly choose which script nodes to express (See the story in the introduction, Section 1), which reflects other work on hierarchical story generation (Fan et al., 2018). At the same time, they exhibit variation in the level of detail. The story in (b) is "shallow" in its representations of the Entering and Leaving scenes and "deep" in its Ordering and Eating scenes. The story in (c), on the other hand, is "shallow" in its representations of the Ordering and Eating scenes and "deep" in its Entering and Leaving scenes. The story in (a) is a result from running HMSSG with a maximum depth of three in all scenes.

In viewing the shape of the the hierarchical script, we draw attention to the fact that, as a tree, there are "deep chains" in which a subtree has a chain of nodes in which each node has two children and the second child is a terminal node with a CD pattern identical to the parent. These would appear to be a consequence of the restaurant being a highly stereotyped and predictable situation. The certainty of the stereotyped activity is what allowed the original scripts to represent knowledge in terms of event chains in which one event creates the preconditions for the next, and this is reflected in a chain pattern in the hierarchical script.

One of the main issues that we encountered when constructing hierarchical scripts was that we attempted to strike a balance between preserving the original script structure and building something akin to a goal hierarchy in which the event present in a child node could be said to be in service of the goal present in its parent node. There were cases where the most abstract maincon from the original script mismatched what we though of as the most abstract goal of the script—maincons were not always identical to subgoals.

If we make the maincon of the exiting scene "C leaves restaurant", then this implies that paying is part of the act of leaving, or that the reason for paying is "so that the customer can leave". This seems true, but only to an extent. It is totally possible to leave without paying. This reflects a finding in Macbeth et al. (2025a) where the real highest-level goals might be the fulfillment of social obligations and "showing appreciation" to the server, which is something that is easy to do. The patron paying is part of the "transaction" in exchange for the restaurant preparing and delivering food, and completing the transaction could be considered a very important and separate goal in relation to eating.

It appears to be important to understand the goals of the other actors involved. Paying the bill is part of appreciating or understanding the goals of the restaurant owner, and of the waiter. It also appears that none of these goals—leaving, tipping, paying the cashier—is more important than any other. Paying first is useful to do before leaving, but if there were a way to pay remotely or run a tab, then one could possibly leave the restaurant and then pay the bill later.

6. Related Work

There is a long and rich history of work on story generation systems (Bertram & Montfort, 2024) which even includes attempts to use hierarchies in generating stories from neural networks (Fan et al., 2018). In this context, it is important to mention TALE-SPIN (Meehan, 1977, 1981), an interactive story-generation system based on problem-solving and cognitive simulation. Unlike

earlier template-filling story generators, it automatically generates stories by simulating characters' goal-oriented behaviors in a fictional world. The characters take actions to address their problems, and an embedded problem solver receives a character's goal and transforms it into subgoals, plans, and actual events and actions. Through its occasional "mis-spun" erroneous tales, TALE-SPIN revealed, in a variety of ways, the importance of commonsense knowledge in story creation, as well as the value of using story generation to test how particular knowledge representations, goal control, and reasoning rules reflect those of human cognition.

Scripts were proposed and initially explored alongside separate plan and goal structures (Schank & Abelson, 1977). Related work that followed focused on how scripts might be learned and generalized through experience (Schank, 1983) and the importance of goals and planning to story understanding (Wilensky, 1983). Later work attempted to extract scripts from large corpora of natural language with information extraction methods (Mueller, 2007), language modeling approaches (Chambers & Jurafsky, 2009; Pichotta & Mooney, 2016), and crowdsourcing (Li et al., 2012; Ciosici et al., 2021). Recently attempts have been made to extract script hierarchies in a limited way in the form of "tracks" (Weber et al., 2018). A script-based bilingual benchmark has been developed for evaluating the social intelligence of large language models (Zhou et al., 2025).

Since the original SAM system lacks the ability to process unexpected scene behaviors, McKenzie et al. (2022) proposed two enhancements of SAM: the MUltiple SCRipt AcTivator (MUSCRAT), and the Script Combination Applier Mechanism (SCAM). MUSCRAT allows multiple scripts to be active during the pattern matching process. SCAM demonstrates its capabilities of producing more complex story narrative situations by combining a restaurant script with a birthday script. Whereas previous work focuses on combining distinct scripts, our work examines how hierarchy within a restaurant script can guide event selection and level of detail in generated stories. Although the original SAM used conceptual dependency structures (Schank, 1972, 1973), a recent effort builds a version of SAM around image schemas (Macbeth et al., 2025b).

The Hierarchical system we propose in this paper bears similarities to hierarchical planning systems like SHOP (Nau et al., 1999) and GTPyhop (Nau et al., 2021). A key distinction, noted in prior work (Macbeth et al., 2025a), is that those systems focus on *plan* synthesis while the one presented here performs *story* synthesis. This leads to slightly different emphases in the overall data structures, procedures, and downstream usages by the story generation engine. However, in principle, further linkages may allow a deeper connection to hierarchical planning systems that future work could explore.

7. Conclusion and Future Work

We have presented our moderate success in creating hierarchical versions of scripts in this paper, as evinced by stories generated from the scripts as well as the scripts themselves. The stories presented may not be interesting because the things that happen are only normal mundane things that happen in restaurants. The TALE-SPIN story generation system (Meehan, 1981) worked by presenting problems to story characters which they attempted to solve with goal and plan structures. From the standpoint of HMSSG as simply a story generator, one area of future work is determining how we present problems to script actors. Scripts in the original SAM had features called *interfer-*

ences which represented minor but anticipated obstacles to the successful completion of the typical sequences of events. One classic example of this is when the customer arrives at the restaurant to find that there are no tables available. Another example is the situation where the customer orders a food item off the menu, but the cook tells the waiter that that item is unavailable, so the waiter goes back to the customer for a different order.

But the ramifications of our study on the evolution of scripts and how evolved views of scripts fit with related conceptual structures more broadly are of greater importance. Script theory (Schank & Abelson, 1977; Cullingford, 1977b) describes scripts as having "tracks" and "subscenes" encompassing variations of the situation or different ways of "accomplishing" the activity. For example, while Schank & Abelson's (1977) detailed description of the restaurant script is a description specifically of the coffee shop track, in which the customer finds their own table, a server takes their order and serves them at their table, and then the customer pays at a cashier, they also wrote of the "fancy restaurant track", the "fast food track", and "cafeteria track" without providing much detail concerning how common structures in tracks are stored or represented. Our work suggests avenues of further exploration in script hierarchies that provide efficient storage and recall of knowledge about the variety of situations in script tracks and subscenes.

Hierarchy creates new opportunities for combining multiple scripts in novel and interesting ways for story generation or story understanding, as also examined by McKenzie et al. (2022). One way of combining scripts is suggested by Cullingford, who wrote of "transitions" between component scripts being handled by more complex global scripts in the $TRIP script (Cullingford, 1977b). Cullingford writes of $BUS, $TRAIN, and $PLANE as "'instrumental' means of reaching or leaving the place where the 'goal' activity of a trip takes place" (Cullingford, 1977b) referencing both the hierarchical nature and the planning nature of scripts.

The conceptual dependency (CD) system also has instrumental cases (Schank, 1975). The instrument case of a CD act is another CD act which is performed in the service of that act. A typical example is "walking" represented as a PTRANS act with a person going from one place to another having an instrumental case consisting of a MOVE act where the person moves their feet and legs. Schank (1975) "define[s] an act as being a series of little actions that make up a whole", and as "a whole combined action". CD acts could be viewed as small scripts, and when scripts are composed of structures consisting of trees or chains of CD acts, this leads to the possibility, to be explored in future work, of a continuum between conceptual dependency and scripts.

References

Abbott, V., Black, J. B., & Smith, E. E. (1985). The representation of scripts in memory. *Journal of memory and language*, *24*, 179–199.

Bertram, L.-Y., & Montfort, N. (2024). *Output: An anthology of computer-generated text, 1953–2023*. Cambridge, MA: MIT Press.

Bower, G. H., Black, J. B., & Turner, T. J. (1979). Scripts in memory for text. *Cognitive psychology*, *11*, 177–220.

Cardona-Rivera, R. E., Jhala, A., Porteous, J., & Young, R. M. (2024). The story so far on narrative planning. *Proc. of ICAPS*, *34*, 489–499.

Chambers, N., & Jurafsky, D. (2009). Unsupervised learning of narrative schemas and their participants. *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP: Volume 2 - Volume 2* (pp. 602–610). Suntec, Singapore: Association for Computational Linguistics.

Ciosici, M., Cummings, J., DeHaven, M., Hedges, A., Kankanampati, Y., Lee, D.-H., Weischedel, R., & Freedman, M. (2021). Machine-assisted script curation. *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Demonstrations* (pp. 8–17). Online: Association for Computational Linguistics.

Cullingford, R. E. (1975). An approach to the organization of mundane world knowledge: The generation and management of scripts. *American Journal of Computational Linguistics*, (pp. 50–69).

Cullingford, R. E. (1977a). Controlling inference in story understanding. *Proceedings of the Fifth International Joint Conference on Artificial Intelligence* (p. 17). Cambridge, MA.

Cullingford, R. E. (1977b). *Script application: Computer understanding of newspaper stories*. Doctoral dissertation, Yale University, New Haven, CT.

Cullingford, R. E. (1979). Pattern-matching and inference in story understanding. *Discourse Processes*, *2*, 319–334.

Fan, A., Lewis, M., & Dauphin, Y. (2018). Hierarchical neural story generation. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 889–898). Melbourne, Australia: Association for Computational Linguistics.

Goldman, N. M. (1975). Sentence paraphrasing from a conceptual base. *Communications of the ACM*, *18*, 96–106.

Lehnert, W. (1975). What makes SAM run? script based techniques for question answering. *Proc. Workshop on Theoretical Issues in NLP* (p. 16–21). USA: Assoc. for Comp. Ling.

Lehnert, W. G. (1978). *The process of question answering: A computer simulation of cognition*. Hillsdale, NJ: Lawrence Erlbaum Associates.

Li, B., Appling, D. S., Lee-Urban, S., & Riedl, M. O. (2012). Crowdsourcing narrative intelligence. *Advances in Cognitive Systems*, *2*, 25–42.

Macbeth, J. C., Roberts, M., Zhang, B., Badhan, S., Neu, M., Garg, T., Hua, Y., Muco, M., & Zhou, M. (2025a). A view of the restaurant script through the lens of hierarchical planning. *Proceedings of The Twelfth Annual Conference on Advances in Cognitive Systems*. Atlanta, GA, USA.

Macbeth, J. C., Zhang, B., & Badhan, S. (2025b). Script-based inferences in an image schema story understander. *Proceedings of The Ninth Image Schema Day (ISD9)*. Cantania, Italy: The International Association for Ontology and its Applications.

McKenzie, M., Kilayko, A., Macbeth, J. C., Carter, S., Sieck, K., & Klenk, M. (2022). Script combination for enhanced story understanding and story generation systems. *Proceedings of The Tenth Annual Conference on Advances in Cognitive Systems*. Arlington, VA, USA.

Meehan, J. (1981). Tale-spin. In R. C. Schank & C. K. Riesbeck (Eds.), *Inside computer understanding: Five programs plus miniatures*, 197–226. Hillsdale, NJ: Lawrence Erlbaum Associates.

Meehan, J. R. (1977). Tale-spin, an interactive program that writes stories. *Proceedings of the Fifth International Joint Conference on Artificial Intelligence* (pp. 91–98). Cambridge, MA.

Mueller, E. T. (2007). Modelling space and time in narratives about restaurants. *Literary and Linguistic Computing*, *22*, 67–84.

Nau, D., Cao, Y., Lotem, A., & Muñoz-Avila, H. (1999). SHOP: Simple hierarchical ordered planner. *Proc. IJCAI* (pp. 968–973). Stockholm: Morgan Kaufmann Publishers, Inc.

Nau, D. S., Roberts, M., Georgievski, I., Geib, C. W., Goldfield, M., & Zhuo, H. H. (2021). GTPyhop: An HTN/goal-task planner written in Python. *ICAPS Workshop on Hierarchical Planning (HPlan)*. Preprint: `https://arxiv.org/abs/2106.02891`.

Pichotta, K., & Mooney, R. J. (2016). Learning statistical scripts with lstm recurrent neural networks. *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence* (p. 2800–2806). AAAI Press.

Schank, R. (1973). The conceptual analysis of natural language. In R. Rustin (Ed.), *Natural language processing*, 291–309. New York: Algorithmics Press.

Schank, R. C. (1972). 'Semantics' in conceptual analysis. *Lingua*, *30*, 101–140.

Schank, R. C. (1975). *Conceptual information processing*. New York, NY: Elsevier.

Schank, R. C. (1983). *Dynamic memory: A theory of reminding and learning in computers and people*. New York: Cambridge University Press.

Schank, R. C., & Abelson, R. P. (1977). *Scripts, plans, goals and understanding: An inquiry into human knowledge structures*. Mahwah, NJ: Lawrence Erlbaum Associates.

Schank, R. C., & Riesbeck, C. K. (1981). *Inside computer understanding: Five programs plus miniatures*. Hillsdale, NJ: Lawrence Erlbaum Associates.

Weber, N., Shekhar, L., Balasubramanian, N., & Chambers, N. (2018). Hierarchical quantized representations for script generation. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 3783–3792). Brussels, Belgium: Association for Computational Linguistics. From `https://aclanthology.org/D18-1413`.

Wilensky, R. (1983). *Planning and understanding*. Reading, MA: Addison-Wesley.

Zhou, J., et al. (2025). SocialEval: Evaluating social intelligence of large language models. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 30958–31012). Vienna, Austria: Association for Computational Linguistics. From `https://aclanthology.org/2025.acl-long.1496/`.

8. Appendix

Below is a table mapping the event CD structures from the restaurant script in Schank & Abelson (1977) to English glosses, organized by scene. The waiter and the "food item" script variables are abbreviated as W and F respectively. For clarity, the customer is abbreviated as C instead of the S which appears in Schank & Abelson (1977). Duplicates of events which appeared more than once in different scenes (e.g. "The waiter goes to the customer's table") have been left out.

Scene	CD	English
Entering	C PTRANS C into restaurant	The customer goes into a restaurant
	C ATTEND eyes to tables	The customer looks at the tables
	C MBUILD where to sit	The customer decides where to sit down.
	C PTRANS C to table	The customer goes to a table.
	C MOVE C to sitting position	The customer sits down at a table.
Ordering	C PTRANS menu to C	The customer picks up the menu
	C MTRANS 'need menu' to W	The customer tells the waiter that they need a menu.
	W PTRANS W to menu	The waiter goes over to where the menus are.
	W PTRANS W to table	The waiter goes to the customer's table
	W ATRANS menu to C	The waiter gives a menu to the customer
	C MTRANS food list to CP(C)	The customer reads the menu
	C MBUILD choice of F	The customer chooses a food item.
	C MTRANS signal to W	The customer signals the waiter.
	C MTRANS 'I want F' to W	The customer tells waiter the food item that they want.
	W PTRANS W to Cook	The waiter goes to the cook.
	W MTRANS (ATRANS F) to C	The waiter tells the cook to give them the food item that the customer ordered.
	Cook MTRANS 'no F' to W	The cook tells the waiter that F is not available
	W MTRANS 'no F' to S	The waiter tells the customer F is not available
	C DO (prepare F script)	The cook prepares the food item that the customer ordered.
Eating	C ATRANS F to W	The cook gives the food item that the customer ordered to the waiter.
	W ATRANS F to S	The waiter gives the food item that the customer ordered to the customer.
	C INGEST F	The customer eats the food item that they ordered.
Exiting	C MTRANS to W (W ATRANS check to C)	The customer asks the waiter for the check.
	W MOVE (write check)	The waiter writes or prints a check for the customer.
	W ATRANS check to C	The waiter gives the customer the check.
	C ATRANS tip to W	The customer gives the waiter a tip.
	C PTRANS C to M	The customer goes to the cashier.
	C ATRANS money to M	The customer gives money to the cashier.
	C PTRANS C to out of restaurant	The customer leaves the restaurant.